

Hybrid Post-Quantum Key Exchange for WireGuard

Integrating ML-KEM into a Production Rust Userspace Implementation

Mihai-Cristian Farcaș

Babeș-Bolyai University
Faculty of Mathematics and Computer Science

Bachelor's Thesis Defense | July 2026

Supervisor: Prof. Dr. Darabant Sergiu Adrian

1. The problem – why WireGuard needs post-quantum key exchange
2. Two approaches – PSK-slot baseline & full hybrid
3. The hybrid handshake, security & implementation
4. Evaluation & conclusions – then a live demo

The problem: a quantum clock, and a protocol that can't be changed

- WireGuard's key exchange rests **entirely on X25519** – which Shor's algorithm breaks once a large quantum computer exists.
- **Harvest-now, decrypt-later:** adversaries record encrypted traffic *today*, decrypt it *later*. For long-lived secrets, migration is urgent *now*.
- NIST finalised **FIPS 203 (ML-KEM)** in 2024; classical key exchange **deprecated by 2030**, disallowed by 2035.

But WireGuard is deliberately rigid

One fixed cipher suite, **no negotiation** – a security feature (kills downgrade attacks), but **no slot to drop a new algorithm into**. Two natural handles: the **PSK slot**, and the **Noise IK handshake** itself.

Research question and two approaches

The question

Can WireGuard's handshake be extended with **hybrid post-quantum key exchange (X25519 + ML-KEM-768)** while preserving its security properties and staying viable on **resource-constrained hardware**?

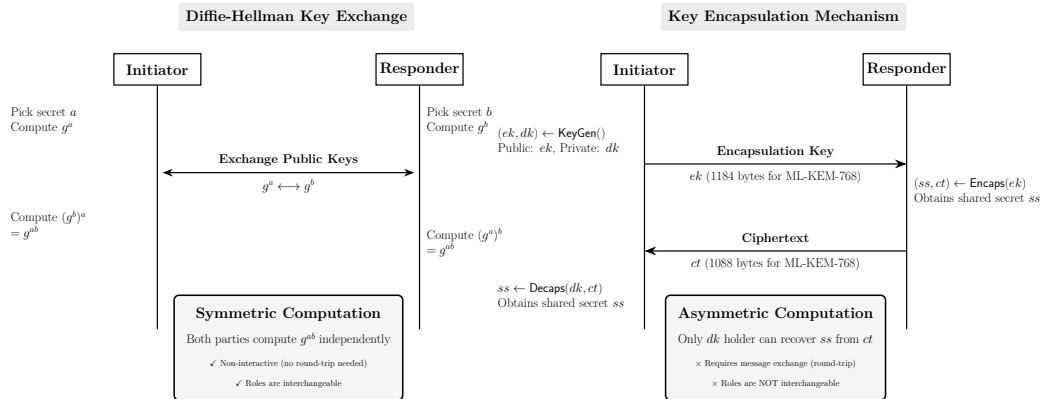
A. PSK-slot baseline

- One-time ML-KEM exchange out-of-band → fill the PSK slot.
- **Zero** WireGuard changes; works with any client.
- The *lower bound* on engineering effort – a controlled comparison point.

B. Full hybrid (the contribution)

- ML-KEM-768 *inside* the Noise IK handshake, every session.
- Fresh KEM keypair per handshake.
- Target: **BoringTun** (Cloudflare's production Rust userspace WireGuard).

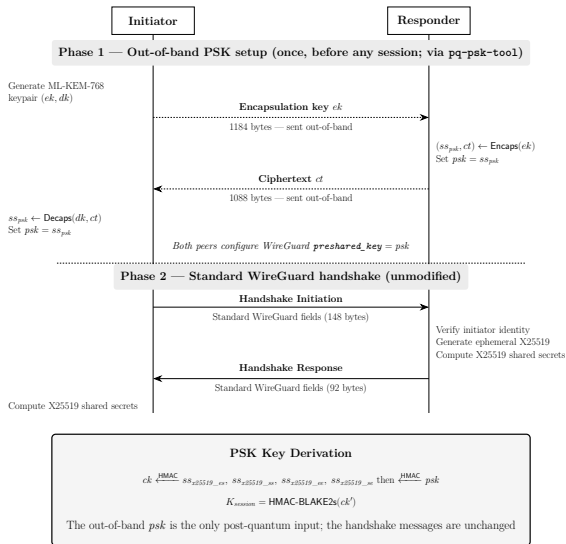
A KEM is not a Diffie–Hellman



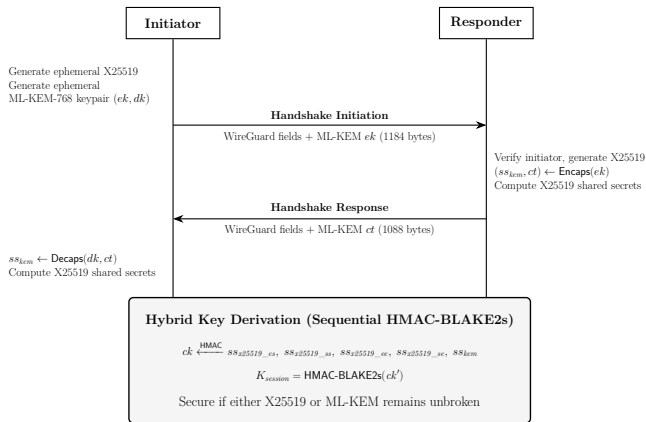
Why ML-KEM-768, why hybrid

- **ML-KEM-768:** The general-purpose default recommended by NIST/IETF; balances size vs security.
- **Hybrid, not pure-PQ:** the MLWE assumption is young; pairing with X25519 means a future lattice break still leaves sessions protected. NIST and IETF recommend hybrid for the transition.
- **vs related work:** PQ-WireGuard is pure-PQ on prototypes with non-standard KEMs; Rosenpass/ExpressVPN carry PQ secrets through the PSK slot. This work modifies the Noise handshake directly, on production-ready code, with standardised ML-KEM – complementary to the formal Hybrid-WireGuard proof (Lafourcade et al.).

The PSK-slot baseline



The hybrid handshake: ML-KEM rides the existing two messages



- **Msg 1:** WG fields + ML-KEM ek (1,184 B)
- **Msg 2:** responder Encaps + ct (1,088 B)
- Initiator Decaps $\rightarrow$ shared ss_{kem} ; **no extra round trips**

Combining secrets: after the four X25519 DHs, one extra HMAC-BLAKE2s step folds in ss_{kem} (WireGuard's *own* KDF); ek/ct bind into the transcript hash.

ek/ct sit *before* the MACs $\Rightarrow$ the anti-DoS cookie covers the PQ payload. **Transport untouched** – throughput stays identical.

Hybrid-secure

The session key is indistinguishable from random if **X25519 or ML-KEM** is secure (sequential chaining / concatenate-then-KDF; HMAC-BLAKE2s as a PRF).

- **Post-quantum forward secrecy:** fresh ML-KEM keypair *per handshake* $\Rightarrow$ past sessions stay safe even if X25519 later falls.
The PSK-slot baseline cannot match this directly: a static PSK, one compromise exposes many sessions.
- **Authentication stays classical** (X25519 static keys) – *deliberate*: it cannot be broken *retroactively*, and NIST prioritises key-exchange migration over signatures.

Implementation

- All PQ code behind a Cargo pq feature flag $\Rightarrow$ **a vanilla build is byte-for-byte identical to upstream.**
- ml-kem crate (RustCrypto): pure-Rust, no_std, NIST KAT-tested $\Rightarrow$ **no C FFI**, runs on the Pi.
- New message types **5/6**; parser dispatches on type+size; a vanilla build cleanly rejects them (InvalidPacket).

Extended BoringTun's own Criterion benches & `noise::tests` – full PQ handshake, end-to-end transport, vanilla-rejects-PQ.

... and we'll see it running in the live demo.

Evaluation – per-primitive & handshake cost (Criterion)

Per-primitive (μs)		
Operation	M4	Pi 5
X25519 KeyGen	8.77	58.00
X25519 DH	20.39	192.81
ML-KEM-768 KeyGen	24.13	81.66
ML-KEM-768 Encaps	20.82	82.77
ML-KEM-768 Decaps	28.03	115.15

Full handshake (μs)		
Configuration	M4	Pi 5
Vanilla	150.8	1312
PSK-slot	151.0	1312
Hybrid	247.3	1643
<i>overhead</i>	+64%	+25%

- **Surprise:** on the Pi one X25519 DH ($193 \mu\text{s}$) is *slower* than any ML-KEM op – x25519-dalek lacks NEON; ML-KEM's NTT maps to the integer pipeline.
- **Size is the real cost:** handshake **240 B** $\rightarrow$ **2,512 B**, yet both messages fit a 1,500 B MTU **in one datagram** (confirmed over Wi-Fi).
- **Footprint negligible:** binary +65 KiB, RSS +0.16 MiB.
- **Throughput identical** – same data plane.

Contributions

1. First FIPS 203-compliant *hybrid* WireGuard on a production Rust codebase.
2. A PSK-slot baseline as a controlled comparison point.
3. A hybrid Noise IK handshake with *post-quantum forward secrecy*.
4. Evaluation across two ARM tiers (Apple M4, Raspberry Pi 5).

Bottom line: hybrid PQ WireGuard is **practical today** – NIST-aligned, userspace-deployable, sub-millisecond overhead, data plane untouched.

Thank you.

Questions?

Mihai-Cristian Farcaş | Supervisor: Prof. Dr. Darabant Sergiu Adrian

Backup – MTU & interoperability

- Hybrid initiation 1,332 B; fragmentation only on paths below $\sim 1,360$ B (e.g. IPv6's 1,280 B minimum).
- **Lowering the WireGuard MTU does *not* help** – that shrinks data packets, not the fixed-size handshake. The fix must live in the handshake (app-level segmentation / PMTUD – future work).
- Interop: a vanilla peer drops PQ types 5/6 at dispatch (`InvalidPacket`); no crash, but no auto-fallback – both peers must be hybrid-capable. Capability negotiation is engineering, not fundamental.

Backup – combining the secrets (key derivation)

One extra mixing step, using WireGuard's own KDF

After the four X25519 secrets are mixed in as usual, fold in ss_{kem} :

$$temp = \text{HMAC-BLAKE2s}(ck, ss_{kem}), \quad ck' = \text{HMAC-BLAKE2s}(temp, 0x01)$$

ek and ct are also bound into the transcript hash h .

- **Reuses WireGuard's HMAC-BLAKE2s** – no new HKDF/SHA-256 introduced; keeps the security argument clean.
- **Transport phase is byte-for-byte unchanged** – ChaCha20-Poly1305 untouched.
This is why throughput is identical.

Backup – throughput & footprint detail

- **Throughput unchanged:** ~470 Mbps loopback, ~80–85 Mbps cross-device; vanilla vs hybrid CIs overlap (same ChaCha20-Poly1305 data plane). Treat the table as a correctness check.
- **Footprint negligible:** binary **+65 KiB** (+5.7%), peak RSS **+0.16 MiB** – a non-issue even on the Pi.
- **Packet size:** handshake grows **240 B** → **2,512 B** (+2,272 B); both messages fit a 1,500 B Ethernet MTU in a single datagram, confirmed over real Wi-Fi, no fragmentation.